Supplementary material for "dinara-align: hardware-aware algorithms for exact pairwise alignment"

Konstantinos Kyriakidis and Benedict Paten

This supplement provides implementation details, per-change measurements and additional results that the main text summarises. Section S1 gives implementation details, both of techniques the main text describes and of others, and how the library was validated (Section S1.7); Section S2 lists the measured effect of each change and of each technique switched off in turn; Section S3 gives the complete results at unit and gap-affine costs, the other alignment modes, the samples, memory and batches; and Sections S4 and S5 describe robustness measures and approaches that were evaluated and not adopted. Unless stated otherwise, the measurements in Sections S1, S4 and S5 and in Table S1 were taken when each change was introduced, on the machine indicated, and compare the change with the preceding version; Table S2 was measured at commit `dc980d7`. The results in Section 4 of the main text and in Section S3 were obtained on the Skylake-X at commit `036e12b`, gap-affine costs at `a6d9166`, and the other alignment modes and batches at `289bcf0`. Timings were taken on an Intel Core i9-7900X (Skylake-X, AVX-512) and an Apple M2 (NEON); Section S1.5 also reports an NVIDIA GeForce RTX 2070. Memory is given in MiB, and references are listed in the main text.

S1 Implementation details

S1.1 Unit costs

Engine selection. The diagonal transition receives a step budget proportional to the cost of the band it would replace; on the Skylake-X a step costs about 1.3 ns and a band column about 6 ns. Inexact seeds replace exact seeds when fewer than 20, 30 or 40% of exact seeds chain on AVX-512, AVX2 and NEON, respectively: the wider the vector unit, the cheaper the bit-parallel band relative to the scalar construction of inexact seeds. All thresholds are fixed at compile time for each instruction set, so that no dispatch occurs at run time.

Register and memory layout of the band. An empty inline-assembly barrier prevents the compiler from reassociating the regrouped recurrence back to its original form. Vertical differences remain in registers across a word's columns, and the horizontal differences of successive columns are stored at distinct addresses, so that no load waits on a preceding store. Bands of five to seven words combine a four-lane vector with scalar words.

Symbols other than ACGT. `N` and other non-ACGT symbols are encoded in a third bit plane. Each tile uses only the planes its symbols require: two when neither its columns nor any row contains such a symbol, the third plane of the rows as a mask when only rows contain one, and all three otherwise. Bases are coded 32 bytes at a time, and symbols and column planes 16 bytes at a time; only chunks that contain such a symbol are processed byte by byte. This made reads containing `N` 7 to 9% faster on both machines. A seed that contains `N` is not counted, so that pairs containing `N` can still use seeds; this made the ultra-long reads 2.5 to 3 times faster on the M2 and 1.4 to 1.7 times faster on the Skylake-X.

Tile width. Tiles are 256 columns wide, as in A*PA2, whatever the height of the band. Earlier versions used tiles of 64 columns in bands bounded below 1,024, a rule kept from the first version on an argument never measured: narrower tiles cut the slope a band adds across each tile, but each tile is pruned, recorded and traced back on its own. With one width, 10 kbp pairs at 5% divergence aligned in 254 µs instead of 318, the 10 kbp nanopore reads in 147 instead of 161 µs, and every other dataset but one in the same time or up to 6.5% faster; 10 kbp pairs at 1%, settled mostly by the diagonal transition, were within 3%.

Traceback within a memory limit. The traceback reads the left boundary of every tile, which the band records; on 1 Mbp pairs at 20% errors, keeping every boundary made an alignment grow peak memory by up to 175 MiB. The band keeps every boundary while they fit within the memory limit of the alignment, 80 MiB by default, and otherwise only every k-th, k doubling as the round proceeds, leaving room for one stretch of the widest boundaries. Each tile also records the state the band carries into it: its words, the score at its top, the deepest kept row and its score, the bound and the checkpoints passed. The traceback, from the last tile back, sweeps a stretch again from the last kept boundary before a tile it lacks; every step of the sweep depends only on that state and the scores it computes, so each boundary is recovered exactly and the alignment is the same byte for byte, unlike the Hirschberg split of QuickEd (Doblas et al., 2025), which breaks ties within each half. Kept boundaries form a stack in tile order and are released as they are traced. The boundary buffers grow by doubling up to half the limit and then to the whole of it at once, since doubling further would hold the old buffer and its copy together, up to twice the limit. On the four 1 Mbp pairs at 20% errors, measured one per process, an alignment grew the peak memory by 133, 98, 124 and 113 MiB instead of 175, 99, 143 and 146 MiB, in 3.64 s instead of 3.35 s for the four, and pairs whose boundaries fit were unchanged. With no memory at all, alignments took about twice the time.

Accepting an extrapolation. Diagonal transition hands over to the band only once its projected work exceeds the band's and it has spent one twenty-fourth of the cheapest band (main text, Section 3.10). An extrapolation sets the band only when the single-wavefront extrapolation from the first edits and the extrapolation of the longer search agree to within one third; otherwise the first bound is set 256 above the known lower bound, and each retry

at most doubles it. A later checkpoint never lowers the bound below the previous bound plus one quarter, and a failed round continues from the bound at which it ended rather than the one at which it started. Before these rules, a 30 kbp SARS-CoV-2 pair at distance 49 had been extrapolated to 14,448 and computed over the full matrix.

When they were introduced, these rules reduced the mean time on the ont-500k-genvar reads from 4.33 s, with 13 of 15 reads completed within the time limit, to 754 ms. At commit `dc980d7`, where Table S2 was measured, the seed heuristic sets the first bound on long reads, and switching the agreement test off changes the ultra-long reads by less than 1% but slows the ont-10k reads by 22% (Section S2.1).

The bottom of the next tile. A tile's rows below the deepest row kept in the previous one can be bounded before the tile is swept: a cell j rows below the diagonal through the deepest kept cell costs at least j more than that cell's score, so the band stops where that plus the heuristic exceeds the bound, as A*PA2 bounds a block.

Retries with seeds. When seeds are used, a retry sets the bound one quarter of the extrapolated increase beyond the estimate; with one half, a round had failed after 90% of the columns of a 527 kbp read. A seeded round that fails within the first eighth of its columns does not extrapolate; instead, its margin over the heuristic bound at the origin increases fourfold, as in A*PA2. Previously, an 884 kbp read had failed at column 1,280 and been extrapolated to 125,000 for a distance of 75,000. Only the first round revises its bound at the checkpoints; when later rounds also lowered their bounds, they failed late, four times on one 897 kbp read.

Traceback. A tile on which the forward search exceeds its limit is recomputed over a window of rows above the traced cell, starting at four words and doubling. The top boundary of the window is initialised to +1, so that its scores correspond to real paths. A 600 kbp nanopore pair at 13% divergence was aligned in 617 ms instead of 1.38 s, of which 840 ms had been spent recomputing tiles of 900 words. Recomputation buffers are reused without initialisation, and tile boundaries are allocated once and written through pointers; loading the boundaries had accounted for a quarter of the traceback time. The CIGAR string is produced by reversing moves 16 at a time and splitting diagonal runs eight bases at a time, which costs 1 to 3% of the alignment time.

Diagonal transition. Wavefronts are padded with unreached diagonals and advanced eight diagonals at a time with branch-free validity masks, in a separate function; a step costs about 1.3 ns, compared with 5 ns before. The two-ended search tests the overlap of the two wavefronts with vector comparisons and requires about half as many steps. Reducing the step budget (Engine selection, above) from 12 to 5 steps per column reduced the time for 10 kbp pairs at 5% divergence from 241 to 144 µs. The choice between one and two wavefronts was fitted on 1,300 pairs of 1 to 30 kbp. For pairs of up to 2,048 columns, the band had been set wide across the whole matrix, and 42% of such pairs were assigned to a band although the diagonal transition would have been faster; such a band is now charged an additional 30,000 steps, which made the 1 kbp reads 25% faster on the Skylake-X. A short noisy pair abandons the diagonal search in favour of the full matrix (a 1 kbp pair at 15% divergence, 19.2 to 11.8 µs), and the first bound of a short pair is 1.7 times its extrapolation, at most 128 above it.

Gather masks. A gather consumes its mask register, and compilers generate the required all-ones mask with `kxnor`, which depends on the previous value of that register; consecutive gathers were therefore serialised at about 21 cycles per group. Deriving the mask from the group's own lanes removes this dependency. Gathered extension made the 1 kbp reads, the SARS-CoV-2 genomes and 100 kbp pairs at 1 to 2% divergence 7 to 19% faster (Table S1, commit 45697e0).

Measured thresholds. A full bit-parallel sweep of a 64-row word costs 35%, 45% and 77% of a diagonal-transition step on AVX-512, AVX2 and NEON, respectively, measured on 1 kbp pairs at 15% divergence (the AVX2 value on the Skylake-X with code compiled for Haswell). On AVX-512, seeds reduce the running time only from about 86 kbp: below that length, the Skylake-X aligned real reads of 16 to 64 kbp 14% faster without seeds, and uniform 30 kbp pairs at 5% divergence 30% faster, whereas at 100 kbp seeds reduced the time by 7 to 24%. With AVX2 and NEON, seeds are used from 16 kbp. For pairs whose two extrapolations agree, as with uniform errors, the inexact-seed threshold is 20% on every platform, and an extrapolated error rate of at least one edit in ten bases selects inexact seeds directly from 128 kbp on AVX-512 and from 64 kbp with narrower vectors: once every band used 256-column tiles, 100 kbp pairs at 10% aligned in 6.8 ms on exact seeds against 10.3 ms on inexact ones on the Skylake-X, while 500 kbp pairs at 10 and 20% ran 7 to 10% slower without inexact seeds.

Every threshold measured again. Each of the 41 thresholds that select an algorithm or bound a search, from the step budgets of the diagonal transition to the seed gates and the traceback's limits, was halved and doubled in turn, and each of the 82 builds was timed on 17 unit-cost samples, with and without traceback, or on 8 gap-affine ones, with the unchanged build timed after every six; its spread was 1.2% in the median. About thirty thresholds moved no sample by more than 3% either way, and most of the others traded one dataset against another, such as the first bound of an untrusted band, or the agreement and give-up rules for extrapolations, and were left as they were. One had become outdated: the length from which inexact seeds are used (Measured thresholds, above). Two more outdated choices were found by examining the slowest results: 64-column tiles for narrow bands (Tile width, above) and the projection that handed nanopore reads to the full sweep (Divergent pairs at gap-affine costs, Section S1.3).

Interleaving. llvm-mca estimates the interleaved AVX-512 loop at 24 cycles per column, against a throughput bound of 11, so the loop remains limited by the lane rotation. A third group still fits in the 32 AVX-512 registers, and llvm-mca predicted 15% fewer cycles per group, but the measured time changed by less than 1.5%.

S1.2 Seed heuristic

Batched lookups. Query rows are processed in batches of 16: the bucket bounds of all rows are loaded first, so that the loads overlap, and their entries are prefetched; each entry stores a seed together with its code, so that a single load provides both. A lookup tests each candidate window with a branch-free expression. Together with considering only seeds that can still chain to the end, these steps reduced the inexact setup on the ont-500k-genvar reads from 5.02 to 2.90 s in total. On 100 kbp pairs at 10% divergence, inexact-seed construction accounted for 58% of the running time on the Skylake-X; there exact seeds now win (Measured thresholds, above).

Admissibility check and correction. Comparing the heuristic with the true remaining cost at every cell of random pairs showed that an earlier version of our implementation overestimated the remaining cost by one on optimal paths in 1 to 2% of pairs. The cause was a filter that discarded matches whose gap to the end of the pair exceeded the potential of the intervening seeds: a match ending one diagonal beyond this limit was discarded, although a path through it costs only one more than the potential, and the neighbour that would otherwise bound such paths could be removed by local pruning. Such a match now chains to the end at the cost of its excess, so an exact match of an inexact seed, which saves 2, starts in layer 1; no further match can follow it. After the correction, no cell exceeded the true cost in 2,200 random pairs, compared with 34,450 cells in 400 pairs before. The reported distances agreed with those of the other aligners in all benchmarks, both before and after the correction. The overestimate arose from our filter; we do not claim that it applies to A*PA or A*PA2.

Lemma S1. Let M hold every match of a seed with cost below its potential r, and let M' be M without the matches whose local-pruning search (Section 2.4 of the main text) finds no path across the next p seeds. The gap-chaining seed heuristic over M' does not exceed the cost of any path from its cell to the end of the pair.

Proof. The heuristic charges a chain of matches the cost of each match and, for each gap between consecutive matches and from the last to the end, the larger of the potential of the seeds that start in the gap and the number of diagonals the gap shifts (Groot Koerkamp and Ivanov, 2024). Let π be a path from cell u to the end, and sweep it from left to right from a column x, at first that of u. Let s be the next seed that starts at or after x; no seed starts between x and s. If π crosses s for at least r, its piece across s pays the potential of s, and the sweep continues from the end of s. Otherwise the crossing is a match $\mu \in M$. If $\mu \in M'$, it joins the chain and the sweep continues from its end. If not, π continued from end(μ) at cost $c(\mu)$ is a path that the search did not find, so its cost reaches the potential of the seeds it has crossed at some column y no later than the end of the p-th seed: the piece of π from the start of s to y pays the potential of the seeds that start in it, and the sweep continues from y. The matches joined appear along π in order, so they form a chain in M'. On each joined match π pays its cost. Each gap is cut into pieces, each of which pays at least the potential of the seeds that start in it and at least the number of diagonals it shifts. Summed over the pieces, π pays in the gap at least the potential of the seeds that start in it and at least the diagonals it shifts, so at least the larger of the two. Hence π costs at least what the heuristic charges this chain, and so at least the heuristic. □

Besides local pruning, the implementation drops only matches whose gap to the end of the pair costs at least what they save, which no chain can use to lower the heuristic; a seed left uncounted, as in a long tandem repeat, has potential 0, which the proof covers. It keeps every other match of M' and some that local pruning would drop (Section 3.5 of the main text); a heuristic over more matches is no larger, so it too never exceeds the remaining cost. The cell-by-cell check above runs with local pruning enabled.

Exact-seed filter. A 32-bit filter per seed, placed before a half-full open-addressing hash table, resolves most lookups with a single predictable test. On the M2, matching a sample of 320 reads took 32 ms instead of 87 ms, and 30 kbp pairs at 5% divergence became 15% faster.

Local pruning. The diagonal-transition search used for local pruning updates its wavefronts in place, with an unreachable diagonal on either side replacing three bounds checks per diagonal.

S1.3 Gap-affine costs

Divergent pairs at gap-affine costs. The wavefront's work grows with the cost times the length, so on unrelated pairs it is slower than an 8-bit difference sweep such as BSAlign's. A global pair with the whole matrix open is therefore handed to a full sweep in difference recurrences (Suzuki and Kasahara, 2018), in 8-bit lanes at any length, once its projected work exceeds the sweep's, at seven sweep cells per wavefront step, the rate measured at 10 kbp. The sweep keeps each cell's predecessor flag under the tie rule of the wavefront, so the alignment is identical byte for byte, and for the traceback it keeps every k-th anti-diagonal, k the square root of the anti-diagonals times the bytes a cell takes, and sweeps each stretch again. Each anti-diagonal is swept in place from the last row up. On unrelated pairs of 1, 2 and 5 kbp at (4, 6, 2) alignment took 0.28, 0.87 and 4.8 ms instead of 1.50, 5.3 and 32.9 ms. The projection extrapolates the cost from how far the two wavefronts have advanced, which on nanopore reads, whose errors gather at the ends, ran up to 2.8 times too high in cost and 8 times in work, and handed reads to sweeps that took up to eight times longer than their wavefront: the ont-10k reads took 2.33 ms instead of 1.56. As for the diagonal transition (main text, Section 3.10), the projection is now trusted only once the wavefront's work has reached one twenty-fourth of the sweep's.

Compact traceback. For the traceback of the wavefront, dinara-align stores, for every cost and diagonal, the

furthest-reaching column in four bytes and one byte of flags recording the predecessor of every layer: five bytes per diagonal, compared with twelve in the high-memory mode of WFA. When the stored wavefronts would exceed the memory limit (80 MiB by default), the pair is split at a cell through which an optimal path passes. Under a general substitution matrix, substitution scores are obtained by byte shuffles (`pshufb`, `tbl`) for matrices of up to 16 entries and by gathers otherwise.

Gathers. In the gap-affine wavefront, AVX-512 gathers reduced the time for 1 kbp reads from 149 to 129 µs. Deriving each gather's mask from the group's own lanes, with one reduction after the loop rather than one per group, reduced it further to 111 µs, and the time for 10 kbp reads from 2.34 to 1.84 ms. Extension along matches was moved into the wavefront step, with columns stored through scalar loads and stores rather than reinserted into a vector, and retained wavefronts are held in fixed blocks that are never moved; memory moves had accounted for 7.5% of profiling samples (Table S1).

Memory layout. The wavefront updates the bounds and furthest reaches of its slots at every cost. When these were separate arrays of a few words, the allocator placed them next to another thread's data, so that the cache line moved between cores at every write; ten threads on the Skylake-X then required twice the cycles per pair of one thread, with 40 cache lines fetched from another core per pair. Each such array now occupies cache lines that no other allocation shares. A worker retains its profile, reversed sequences and search buffers between pairs, so that a warm worker performs no allocation; per-pair allocations had accounted for one third of the time of a short read, and caused contention in the allocator with many threads. Padding each row of a wavefront buffer by one cache line, to avoid 4 KiB aliasing between loads and earlier stores, removed 1.5 billion blocked loads over ten 100 kbp pairs. Separating the working memory of threads by two cache lines rather than one reduced the cache lines fetched from other cores from 3.4 million to 0.3 million over 2 million reads on ten threads.

S1.4 Other modes

Extension. Extension alignment uses a single wavefront search from the anchor, with the match score folded into the costs, and records the best end point on the furthest anti-diagonal of each wavefront. The search stops once a proven bound shows that no higher cost can yield a better score, and the prefix up to the best end point is then aligned globally. With an end bonus, the search also records the best point on the last row of the query and continues only while such an alignment can still come within the bonus; this reduced the time for an extension with an end bonus from 1.02 ms to 186 µs.

Ends-free alignment. Under gap costs with free ends, the forward search starts at cost zero on every diagonal on which a free start lies. The span is determined first: one search from the free side finds the end on the highest diagonal, and a narrow search backwards finds the start; the span is then aligned globally. On x86, this aligns a read at the start of a reference, an overlap or a suffix 1.4 to 3.5 times faster, and a read anywhere within the reference 0 to 14% slower, than the previous approach.

Infix and prefix alignment at unit costs. Infix alignment uses the bit-parallel sweep with the cut-off of Edlib, with a first bound of 64; a failed try extrapolates the distance from the deepest row its band kept within the bound and the retry aims past it, which on the M2 took a 10 kbp infix from 3.39 to 1.99 ms; words entering the band are initialised to +1, and the last word advances one column at a time. Prefix alignment sweeps only the band permitted by its bound: the top of the band moves down with the diagonal, no column beyond the read length plus the best end so far is computed, an attempt whose band becomes empty stops, and at one eighth, one quarter and one half of the columns an attempt that can no longer succeed is abandoned. This reduced the time for a prefix distance from 23.7 to 20.5 µs on the Skylake-X.

A band derived from the score. A table with one match and one mismatch score is converted to costs and aligned by the wavefront (main text, Section 3.8). Under any other substitution matrix, a global alignment is traced in a band of diagonals that grows at most fourfold per step until its own score proves that it holds every optimal path, each cell's decision kept in one byte. With $\alpha_{\max}$ the table's largest score, each gapped letter adds at least $2e + \alpha_{\max}$ to $C = \alpha_{\max}(n + m) - 2S$, and a path that strays t diagonals beyond those between the main and the end diagonal δ has at least $|\delta| + 2t$ gapped letters, so the score of any alignment confines every optimal path to $t \leq (C/(2e + \alpha_{\max}) - |\delta|)/2$. A pair close to its diagonal thus costs about its own cells: with sweeps in 16-bit lanes and one-byte traceback decisions, 1 kbp pairs at 10% divergence under a transition-transversion table aligned in about 200 µs instead of 544 µs on the Skylake-X (Table S1). Anti-diagonals are swept in strips of rows of at most 16 KiB, which stay in the first-level cache; local alignment of 10 kbp against 12 kbp took 17.4 ms instead of 50.0 ms. The backward sweep from the end of a local alignment and each half of a linear-space alignment also use 16-bit lanes while the scores fit, which made 1 kbp local alignments 1.2 times and linear-space global alignments 1.4 times faster on the M2.

S1.5 Batches

Letter layout. The pairs of a group are interleaved so that one 64-bit load reads eight letters of a lane, and eight lanes are transposed in registers by two shuffles.

Alignment in 8-bit lanes. Batch alignments remain in 8-bit lanes, 64 pairs per AVX-512 register, and a pair whose cost exceeds the 8-bit range is passed to a single call. With 16-bit lanes, half as many lanes would sweep the wide band that such a pair requires, which on the Skylake-X took as long as the searches on 1 kbp reads at 10% divergence, and twice as long on NEON.

Cache-efficient traceback. Storing the traceback flags of a group's whole band would exceed the caches on long pairs and touch new memory pages for every group; with ten threads, this doubled the time for 1 kbp pairs relative to the searches alone. The sweep therefore stores one row of every layer every s rows and recomputes a segment to obtain its flags during the traceback. With R rows, s is the square root of R times the bytes per cell of a stored row divided by those of a flag, which balances the stored rows against the flags of one segment and minimises memory use.

Lane groups. A group of 64 bytes occupies one register on AVX-512 and several on narrower vector units; with the 128-bit registers of the M2, 1 kbp pairs at 10% divergence required 73 µs per pair under gap-affine costs with four registers, compared with 132 µs with one. Local scores use 32 lanes of 16 bits or 16 lanes of 32 bits, which on 128-bit registers spreads a group over four independent registers; on the M2, this scored 20,000 references against a read in 83 ms, compared with 160 ms with one register. Pairs whose first band does not satisfy the bound are grouped by power-of-two classes of the width of their second band, and a group stops once every lane has exceeded its cost limit.

Work shares. Batch calls asked for more than one thread spread their pairs over them, longest first: the pairs are dealt into classes of lengths within an eighth of each other, with no sort, since sorting the short reads on one thread had taken a fifteenth of the work ten threads did on them. Each worker then takes a share of half of what is left divided among the workers, as OpenMP's guided schedule does, but never more than twice its last share, starting from one; a first share of a twentieth of the batch had left one worker holding the longest pairs while the others finished, adding half the batch's time on the Skylake-X. Building the packed batch is split across the threads in the same way; on one thread it had taken three times as long as the GPU took to score it.

Free ends in lanes. Ends-free alignments take the lanes too, their span found first as the tie rule picks it for a single pair: first the end, the cell furthest along the reference on which an optimal alignment may stop, found by a sweep whose band provably holds every optimal path; then the start, by a sweep back from that end over the reversed prefixes; then the global alignment of the span. On one core of the Skylake-X, 150 bp reads at 3% errors in 200 bp windows aligned at (4, 6, 2) in 8.0 µs per pair as an infix, against 26.6 µs one pair at a time, in 12.1 against 29.3 µs as a prefix, and in 20.0 against 50.3 µs with 60 letters free at the end of the window and the start of the read; at unit costs that overlap took 8.3 against 32.4 µs.

GPU scores. Batch scores and alignments under a substitution table, and unit-cost distances, also run on NVIDIA GPUs and Apple silicon. Each pair is first scored over a band of sixteen diagonals by one thread, its scores and gap layers one register per diagonal, swept anti-diagonal by anti-diagonal; the band's score stands where the pair's own score certifies it, by the bound of Proposition 1 of the main text with the table's least gap penalties, and the thread stops once the cheapest cell of its anti-diagonal costs more than every path off the band. The remaining pairs, about one in six of the short-read batch, are scored over the whole matrix while the host packs the next chunk, several to a warp, each lane holding a run of its pair's columns in registers and passing its last column on by shuffle. The cells of a global alignment are held shifted by their anti-diagonal times the gap extension, which reduces Gotoh's recurrence from five additions per cell to three. On an NVIDIA GeForce RTX 2070, the 500,000 short-read pairs were scored in 36 ms at (1, 2, 1), the whole call included and most of it the host packing the batch, against 27 ms on ten cores of the Skylake-X.

S1.6 Code generation

Avoiding division by a variable. A 64-bit division by a variable takes tens of cycles on x86. The potential of the heuristic, required at every row that the band prunes, involves a division by the seed length, so each seed length has its own code path with a constant divisor, which compiles to a multiplication; batches likewise avoid a division per pair.

Kernels compiled separately. When inlined next to each other, or selected per band round, the bit-parallel kernels for plain bases and for additional symbols slowed the plain kernel by 1 to 4%; when inlined into the seed construction, the local-pruning search slowed the construction loop by a few per cent. Separate seeded and unseeded versions of the band keep the code of the heuristic out of unseeded pairs, which it had slowed by 3 to 7%.

Operand order. Under a substitution matrix, the recurrence first takes the maximum of the two gap layers, so that the substitution score, the operand that arrives last, waits on only one maximum; a score obtained by comparison is available early and is used first.

Branches. On a 1 Mbp pair at 15% divergence, branch mispredictions account for about 7% of the cycles, mostly at short loop exits, compared with 2 to 3% in the bit-parallel sweep. A fully branch-free seed scan would spend about 25 million cycles per pair to save about 34 million, a gain of 1 to 2%, and was not adopted.

S1.7 Validation

Differential fuzzing. `pixi run fuzz` draws random pairs with costs, modes, bands, cost limits and memory limits, three trials in ten with a small memory limit and every sixteenth as part of a batch, and checks each answer against a full-matrix implementation (`tests/oracle.mojo`) that shares no code with the library: the alignment must be valid and its cost or score optimal. An alignment under a cost limit must cost the same as without one, and the tie rule is checked at the default memory limit, since a pair split to fit a smaller one follows the rule within each piece: there the alignment under a cost limit must be the same as without one, and the default rule must give WFA2-lib's alignment of both sequences reversed, read backwards. A unit test shows that a unit-cost alignment is the same byte for byte under any memory limit. At commit `691f43a`, 20,000 random cases and 400 long ones passed.

WFA2-lib's regression set. `pixi run test-wfa` aligns the 305 pairs that WFA2-lib ships with its tests in every mode it tests, and compares each score with WFA2-lib's and each CIGAR, under WFA2-lib's tie rule, with the one WFA2-lib reports.

S2 Measured effect of each change

Table S1: Changes behind the results and their measured effects, as recorded when each was made. Each row times what its commit timed (an in-process loop through one aligner, the benchmark runner, a distance or an alignment), so rows for one dataset need not agree with each other or with Section 4 of the main text. Machines: the i9-7900X (Skylake-X, AVX-512) and the Apple M2 (NEON); "not recorded" where the commit does not say.

Commit	Change	Measured effect	Machine
a6101e5	Inexact seeds (one edit), matched by halves	100 kbp 15.6 → 9.5 ms; 300 kbp 105 → 47 ms; 1 Mbp 1.36 → 0.41 s	not recorded
7435edd	Leaner seed setup: branch-free test, reachable seeds only, fronts in place	inexact setup on genvar 5.02 → 2.90 s; genvar 11.77 → 10.21 s; ont-500k 8.04 → 7.11 s	Skylake-X
28b9e67	Inexact-seed cut-off by vector width (20% on AVX-512)	ont-500k (50 reads) 8.73 → 8.03 s; ont-500k-genvar (48 reads) 12.03 → 11.73 s; 100 kbp at 6% 114 → 64 ms	Skylake-X
708f151	Seeded retry a quarter of the projected increase past the estimate	ont-500k-genvar 9.86 → 9.66 s; ont-500k 6.81 → 6.30 s; 100 kbp at 15% 4% faster	Skylake-X
ff26cea	Seed gate at 86 kbp on AVX-512; exact-seed filter	30 kbp at 5% 1.48 ms → 898 µs; SARS-CoV-2 373 → 283 µs; ont-50k 897 → 800 µs (recorded in cae258b)	Skylake-X
ff26cea	Exact-seed filter alone	matching a sample of 320 reads 87 → 32 ms; 30 kbp at 5% 15% faster	M2
45697e0	AVX-512 gathers in the diagonal transition	ont-1k 26.8 → 25.0 µs; SARS-CoV-2 335 → 306 µs; 100 kbp at 1% 1.70 → 1.45 ms, at 2% 5.60 → 4.54 ms	Skylake-X
2c7c46e	Short pairs stay in the diagonal transition	ont-1k 42.3 → 31.8 ms over 1,221 reads	Skylake-X
e0ea104	Alignment step budget 12 → 5 a column	10 kbp at 5% 241 → 144 µs	not recorded
a593ade	Short noisy pairs give up the diagonal search	1 kbp at 15% 19.2 → 11.8 µs	Skylake-X
beb7a8c	Tiles traced by a forward search	ont-50k 3% and ont-500k 8% faster	M2
5a0e514	Tile search eight diagonals at a time, gathered	100 kbp at 15% tile searches 3.55 → 1.4 ms a pair	Skylake-X
7f1a813	Windowed recompute of a failed tile	600 kbp at 13% 1.38 s → 617 ms	not recorded
3becb4b	Recompute buffers reused, uninitialised	ont-1k 23.4 → 22.6 µs; ont-10k 151 → 142 µs	not recorded
e2125b0	Tile edges sized once, written through pointers	ont-500k 2 to 3%, ont-10k about 2% faster	both
38837f2	Fronts sixteen offsets a store; unchecked reads	ont-1k 21.4 → 20.7 µs; SARS-CoV-2 213 → 199 µs	Skylake-X
34cb623	Pointers in hot lookups	genvar 10.16 → 9.90 s; ont-500k 7.04 → 6.82 s	Skylake-X
f29a945	Separate seeded and unseeded band copies	unseeded pairs back within 2% (had slowed 3 to 7%)	Skylake-X
aca56c7	AVX-512 gathers in the gap-affine step	ont-1k 149 → 129 µs; ont-10k 3.52 → 2.94 ms; 10 kbp at 15% 36.4 → 24.5 ms	Skylake-X
53e15d4	Gather mask from the group's own lanes; one reduction after the loop	ont-1k 129 → 111 µs; ont-10k 2.34 → 1.84 ms; 10 kbp at 15% 17.9 → 14.0 ms	Skylake-X
ced0b8d	Ring rows padded against 4 KB aliasing	ont-10k 1.86 → 1.68 ms; 100 kbp at 5% 349 → 305 ms	Skylake-X
17fd781	Slides straight from the step	100 kbp at 5% 315 → 287 ms; ont-10k 1.80 → 1.67 ms	M2
01c1e9b	Slide columns through scalar loads and stores	100 kbp at 5% 287 → 271 ms; ont-1k 91 → 86 µs	M2
72c3696	Kept fronts in fixed blocks, never moved	ont-10k 1.48 → 1.37 ms (memmove had been 7.5% of samples)	M2

Table S1: (continued)

Commit	Change	Measured effect	Machine
b7e5232	Gap-affine traceback through the wavefront's own fronts	SARS-CoV-2 1.34 ms → 638 µs; ont-10k 14.6 → 4.05 ms	M2
e6a435f	End bonus weighed in the extension's own search	with an end bonus 1.02 ms → 186 µs (KSW2 1.23 ms)	not recorded
68561b1	Free ends: span first, start at cost zero on free diagonals	start, overlap, suffix 1.4 to 3.5× faster; anywhere 0 to 14% slower	Skylake-X
3c0eb81	Unit-cost infix: Ukkonen's cut-off, restarted words	10 kbp in 100 kbp 21 → 13 ms; 100 kbp in 1 Mbp 2 → 0.75 s	M2
2f8b935	Prefix search abandons a try that cannot succeed	distance 23.7 → 20.5 µs; alignment 45.8 → 42.9 µs	Skylake-X
2b038c8	Global band bounded by the score (one match, one mismatch score)	1 kbp at 5%: 137 of 1,000 diagonals; 0.36 → 0.21 ms	not recorded
756f3e1	Table sweeps vectorised, any table	1 kbp global 13.3 ms → 550 µs; local 133 → 2.89 ms	Skylake-X
433a8b5	Local under gap costs: lanes along the shorter sequence, extension back	against SSW: 47 vs 55 µs on short pairs; 1.93 vs 3.22 ms for 1 kbp in 10 kbp	Skylake-X
cf87b2b	Local sweep in 16-bit lanes	3.4 ms, from 4.6	Skylake-X
a8355aa	Batch letters transposed in registers	gap-affine 0.59 → 0.43 µs a pair; unit 0.55 → 0.40 µs	not recorded
20425a7	Batch first pass in saturating bytes	gap-affine 0.43 → 0.40 µs a pair	Skylake-X
64f0cc3	Second-pass bands grouped by exact width	1 kbp at 10% scores 18 → 15 µs; 46 → 38 µs at (4, 6, 2)	not recorded
0f7b5af	A lane group stops once every lane has passed its cap	1 kbp at 10% aligns 33 → 22 µs, scores 18 → 12 µs	Skylake-X
792ba9f	Pairs with N take seeds; a seed with N goes uncounted	500 kbp reads 2.5 to 3× faster (M2), 1.4 to 1.7× (Skylake-X)	both
a329af5	Table sweeps in 16-bit lanes where the straight path bounds their scores	table-global 544 → 352 µs	Skylake-X
0395d7e	Banded traceback keeps each cell's decision in a byte	table-global 352 → 253 µs	Skylake-X
c5aee28	Table global alignment traced in bands that prove themselves	table-global 243 → 157 µs	M2
691f43a	Divergent gap-affine pairs handed to a full sweep in difference recurrences	unrelated 1, 2, 5 kbp at (4, 6, 2) 1.50, 5.3, 32.9 → 0.28, 0.87, 4.8 ms	Skylake-X
64be0ec	Anti-diagonal sweeps a 16 KiB strip of rows at a time	local, 10 kbp against 12 kbp 50.0 → 17.4 ms	Skylake-X
ffbf46e	Banded traceback, local end search and linear-space halves in place	divergent table pairs 18 to 37% faster	Skylake-X
94bf719	Unit-cost traceback within the memory limit, stretches swept again	1 Mbp at 20%: peak growth 175 → 133 MiB a pair; 3.35 → 3.64 s for four (with 5742ef7, 92deb39)	Skylake-X
289bcf0	Gap-affine hand-over only after one twenty-fourth of the sweep's work	ont-10k at (4, 6, 2) 2.33 → 1.56 ms; ont-50k 30.5 → 21.0 ms	Skylake-X
ea3f09e	256-column tiles for every band	10 kbp at 5% 318 → 254 µs; ont-10k 161 → 147 µs	Skylake-X
dc980d7	Inexact seeds from 128 kbp on AVX-512	100 kbp at 10% 10.3 → 6.8 ms	Skylake-X
7aab823	A space holds its profile directly, not in an Optional, and an aligner's traceback reuses it	250 bp Illumina, through one aligner 0.97 → 0.80 µs; distance 1.21 → 0.68 µs	Skylake-X
90e0040	An aligner keeps the fronts it traces back	250 bp Illumina 0.82 → 0.65 µs	Skylake-X
32e78aa	A gap-affine alignment begins with the one search it traces	250 bp Illumina at (4, 6, 2) through one aligner 2.41 → 1.52 µs (a fresh call 5.16 µs before)	Skylake-X
7d38a99	A reversed pair coded back to front in the coding pass	250 bp Illumina 0.649 → 0.583 µs	Skylake-X
4558186	Alignments pass between internal calls without an Optional	250 bp Illumina 0.585 → 0.420 µs; at gap-affine costs 0.995 → 0.840 µs	Skylake-X
036e12b	A profile coded 32 bytes at a time	250 bp Illumina 0.421 → 0.400 µs; PacBio HiFi 5.7 → 4.95 µs	Skylake-X
a6d9166	The first gap-affine search grows alone while it steps no more than two searches would	250 bp Illumina at (4, 6, 2) 1.26 → 1.20 µs; pairs costing 9 to 32 a third less	Skylake-X

S2.1 Effect of each technique

To measure what each technique contributes to the final code, dinara-align was built with one technique switched off at a time, and every build aligned samples of the A*PA2 real datasets and of the 100 kbp simulated pairs and scored the short-read batch, the builds alternating within three rounds on one pinned core and assembled with branches kept clear of 32-byte boundaries, which the Skylake-X penalises (Table S2); every build returned the same costs. Without direct neighbour construction (main text, Section 3.5), the ultra-long reads took 9% and 16% longer; 100 kbp pairs, which take exact seeds on AVX-512, do not use it. Without the interleaved groups (Section 3.2), the ultra-long reads took 4.5 to 5% longer, 100 kbp pairs at 10% and 20% errors, whose bands are widest,

Table S2: Effect of each technique: median time with all techniques on (ms per alignment; per batch of 500,000 pairs for the short reads), and the change when one technique is switched off, over three alternating rounds on the Skylake-X at commit `dc980d7`, with branches kept clear of 32-byte boundaries. Every configuration returned the same costs.

Dataset	All on	Regrouped recurrence	Interleaved groups	Gathered extension	Agreeing estimates	Direct neighbours	Certified bands
ont-500k	98.8	+1.1%	**+4.5%**	+0.5%	−0.2%	**+8.9%**	
ont-500k-genvar	136.0	+1.9%	**+5.0%**	+0.4%	0.0%	**+16.4%**	
ont-1k	0.021	+0.2%	+0.3%	**+14.9%**	0.0%	+0.7%	
ont-10k	0.145	−0.6%	+1.2%	**+6.4%**	**+22.2%**	+0.1%	
SARS-CoV-2	0.197	−0.9%	+0.2%	**+13.4%**	**+4.9%**	−0.5%	
100 kbp, 1%	1.10	−1.4%	+0.4%	**+26.5%**	−0.1%	−1.0%	
100 kbp, 5%	4.61	−0.6%	−0.2%	+0.1%	0.0%	+0.2%	
100 kbp, 10%	6.72	+0.3%	+3.4%	+1.9%	+0.5%	−0.2%	
100 kbp, 20%	15.0	**+4.1%**	**+11.5%**	+3.9%	−2.9%	0.0%	
Short reads, gap-affine	232						**+188%**
Short reads, unit	213						**+215%**

3.4% and 11.5%, and the other samples within 1.2%. The regrouped recurrence changed the samples by −1.4 to +4.1%, as bands narrow enough to be bound by a single word's step are a small part of the work. Without gathered extension (Section 3.4), the samples on which diagonal transition does most of the work took 6.4 to 26.5% longer, and without the agreeing estimates (Section 3.3), the ont-10k reads took 22% and the SARS-CoV-2 genomes 4.9% longer. Without the certified bands (Section 3.6), the short-read batch took 2.9 and 3.2 times as long.

S3 Additional results

Table S3 gives the results for the other alignment modes, each compared on one pinned thread of the Skylake-X with the aligners that support the same mode. Table S4 gives the complete gap-affine comparison at costs (4, 6, 2). Table S5 gives the mean and median time per alignment at unit costs on every dataset, Table S6 the samples, Table S7 the memory and Table S8 the short-read batch. The approximate aligners of the A*PA2 benchmark, WFA-adaptive and Block Aligner, are not evaluated, as they are not exact. At unit costs, KSW2 and BSAlign, which ran out of memory or crashed on every dataset of 500 kbp and more, are in the benchmark's own tables (benchmarks/README.md in the repository); Table S4 includes them.

S4 Robustness

Three changes address inputs on which the running time had been disproportionate:

- **Low-complexity sequence.** Wavefront layers that spread into a staircase pattern are handled explicitly, and the pruning search is bounded; a poly-A sequence of 32,000 bases aligned against one of 16,000 bases took 0.15 s instead of 17 s.
- **Repetitive seeds.** Seeds are abandoned when they exceed 64 matches per seed on average; setting up a 200 kbp tandem repeat took 0.045 s and 34 MiB instead of 73 s and 760 MiB.
- **Unreachable wavefronts.** A wavefront that reaches no cell is not tested further; one such case took 13 ms instead of 1.5 s.

S5 Approaches evaluated and not adopted

- **Multi-threading within a pair.** Striped bands over eight threads aligned 1 Mbp pairs at 15% divergence in 575 ms, compared with 1,122 ms on one thread. The implementation was removed because a single alignment runs on the calling thread, and distributing pairs over threads, which batch calls do when asked, yields a larger gain.
- **One sixteen-lane group instead of two eight-lane groups.** Equal on long reads, and up to 2% slower on short divergent pairs.
- **A Bloom filter for inexact keys.** It halved the number of candidates, but reduced the alignment time by only 1.5 to 2%.
- **Shorter look-ahead, or no local pruning.** Without pruning, the ultra-long reads with and without genetic variation were 21% and 72% slower, respectively.
- **A* search within the diagonal transition, and an upper bound from beam search.** Each required several tuned parameters for small gains, and A* within the diagonal transition was 3 to 5 times slower on real reads.

Table S3: Other modes: mean time per pair in ms on one pinned thread, the faster of two passes. Every tool's costs or scores agree on every pair.

Workload	dinara-align	Next fastest
Infix at unit costs, 1 kbp read at 10% in a 3 kbp window	**0.132**	Edlib 0.273
Infix at unit costs, 10 kbp read at 5% in a 30 kbp window	**2.01**	Edlib 13.6
Prefix at unit costs, 1 kbp read against 2 kbp	**0.044**	WFA2-lib 0.047
Infix at (4, 6, 2), 1 kbp read in a 3 kbp window	**3.93**	WFA2-lib 3.94
Extension, match 2 at (4, 6, 2)	**0.734**	KSW2 2.06
Extension with an end bonus of 50	**0.166**	KSW2 1.23
Two-piece gaps (4, 6, 2, 24, 1), 5 kbp at 5%	**8.30**	WFA2-lib 15.9
Table, global, 1 kbp at 10%	**0.198**	parasail 2.92
Table, local, 1 kbp read in a 10 kbp window	**1.91**	SSW 3.27
Local, 600 short noisy pairs (20–700 bp)	**0.032**	SSW 0.051
Local, 1 kbp read at 10% in a 10 kbp window	**1.36**	SSW 3.15
Local, 10 kbp at 5% against 12 kbp	**17.5**	SSW 85.1
Overlap of 2 kbp reads by 0.5–1.5 kbp	**0.724**	parasail 15.4
Infix with a reward, 1 kbp read in a 3 kbp window	**0.693**	parasail 15.2

- **Traceback during the sweep.** On the ultra-long reads and on 1 Mbp pairs at up to 15% errors, the stored tile boundaries occupied 0.6 to 13 MiB, compared with 10 to 17 MiB for the sequences and bit planes, so little memory could be saved; the traceback within a memory limit (Section S1.1) now bounds the worst case.
- **A tighter first bound.** With each pair's distance supplied by an oracle, starting band doubling at it saved up to 27% (100 kbp pairs at 20% errors), but starting 15% above it lost 18 to 23% on the ultra-long reads; capping the bound at an upper bound 5% above the distance saved up to 27% (1 Mbp pairs at 20%), and one 15% above about half that or nothing. No upper bound cheap and tight enough was found: beam search cost more than it saved, and a batch edit-distance certificate proved 578 of 70,519 pairs.

Table S4: Gap-affine alignment at costs (4, 6, 2), all datasets: mean time per alignment in ms, with a 30 s budget per sample. Brackets give the pairs completed where not all were; t.o. marks a tool that completed none within 30 s and o.o.m. one the 8 GiB limit stopped. Bold marks the lowest mean of the tools that completed the most pairs.

Dataset	dinara-align	WFA	BiWFA	BSAlign	KSW2
Real data					
ont-1k	**0.105**	0.188	0.308	0.336	1.10
ont-10k	**1.60**	4.89	5.86	7.84	30.0
ont-50k	**21.5**	63.4	54.9	87.0	341 [81/104]
ont-500k	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
ont-500k-genvar	22,000 [1/4]	o.o.m.	t.o.	o.o.m.	o.o.m.
SARS-CoV-2	**0.710**	2.05	1.78	341	87.9
Illumina	**0.00145**	0.00155	0.00281	0.0539	0.0128
PacBio HiFi	**0.0950**	0.227	0.231	86.0	9.72
Simulated: random sequences, uniform errors (QuickEd)					
10 kbp, 1%	**0.199**	0.416	0.646	37.0	21.3
10 kbp, 5%	**2.32**	7.14	8.72	37.0	96.1
10 kbp, 10%	**7.06**	23.1	27.1	37.3	202
10 kbp, 20%	**20.1**	66.0	75.1	37.4	375 [81/99]
100 kbp, 1%	**11.9**	39.9	39.0	o.o.m.	2,420
100 kbp, 5%	**331**	808	714	o.o.m.	9,170 [3/9]
100 kbp, 10%	**1,160**	o.o.m.	2,430	o.o.m.	17,300 [1/10]
100 kbp, 20%	**3,610 [8/10]**	o.o.m.	7,360 [4/10]	o.o.m.	o.o.m.
500 kbp, 1%	**477**	1,290	942	o.o.m.	o.o.m.
500 kbp, 5%	**9,910 [3/4]**	o.o.m.	19,400 [1/4]	o.o.m.	o.o.m.
500 kbp, 10%	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
500 kbp, 20%	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
1 Mbp, 1%	**2,300**	o.o.m.	3,940	o.o.m.	o.o.m.
1 Mbp, 5%	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
1 Mbp, 10%	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
1 Mbp, 20%	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.

Table S5: Unit costs, all datasets: mean time per alignment in ms, with the median in parentheses, traceback included, a 30 s budget per sample. Brackets give the pairs completed where not all were, the median over those; t.o. marks a tool that completed none within 30 s and o.o.m. one the 8 GiB limit stopped. Bold marks the lowest mean of the tools that completed the most pairs.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	A*PA	Edlib	BiWFA	WFA	QuickEd
Real data								
ont-1k	**0.0211 (0.0213)**	0.0806 (0.0890)	0.0480 (0.0539)	0.612 (0.556)	0.124 (0.118)	0.0443 (0.0463)	0.0311 (0.0320)	0.0748 (0.0728)
ont-10k	**0.156 (0.109)**	0.391 (0.264)	0.247 (0.177)	15.3 (3.29)	1.09 (0.569)	0.741 (0.278)	0.599 (0.225)	0.688 (0.451)
ont-50k	**0.580 (0.229)**	1.27 (0.513)	0.944 (0.307)	263 (9.26)	6.14 (2.05)	7.43 (0.872)	6.38 (0.726)	2.31 (1.05)
ont-500k	**118 (73.6)**	283 (110)	687 (398)	t.o.	5,000 (3,050)	6,330 (2,190)	o.o.m.	847 (585)
ont-500k-genvar	**156 (156)**	225 (196)	511 (564)	t.o.	5,230 (5,200)	6,200 (6,630)	o.o.m.	1,390 (1,320)
SARS-CoV-2	**0.286 (0.146)**	2.00 (2.07)	0.728 (0.662)	6.49 (2.07)	8.18 (7.74)	0.909 (0.456)	0.987 (0.362)	3.90 (4.27)
Illumina	**0.000576 (0.000482)**	0.0145 (0.0143)	0.00410 (0.00405)	0.0137 (0.0132)	0.0233 (0.0232)	0.00204 (0.00179)	0.00112 (0.000999)	0.0165 (0.0154)
PacBio HiFi	**0.0126 (0.00527)**	0.977 (0.921)	0.259 (0.237)	1.05 (0.962)	3.35 (3.14)	0.0861 (0.0691)	0.0659 (0.0470)	0.950 (0.881)
Simulated: random sequences, uniform errors (QuickEd)								
10 kbp, 1%	**0.0258 (0.0228)**	0.729 (0.724)	0.160 (0.156)	0.564 (0.560)	3.22 (3.21)	0.0925 (0.0883)	0.0657 (0.0576)	0.611 (0.609)
10 kbp, 5%	**0.267 (0.242)**	0.797 (0.794)	0.288 (0.285)	1.08 (1.07)	3.79 (3.80)	0.642 (0.635)	0.508 (0.495)	1.25 (1.24)
10 kbp, 10%	**0.355 (0.361)**	0.726 (0.714)	0.450 (0.439)	3.26 (3.23)	4.56 (4.55)	2.26 (2.26)	1.81 (1.66)	1.99 (2.01)
10 kbp, 20%	**0.512 (0.468)**	1.44 (1.44)	1.02 (1.01)	205 (203)	5.93 (5.93)	7.33 (7.32)	5.32 (5.11)	1.55 (1.56)
100 kbp, 1%	**1.50 (1.11)**	7.53 (7.27)	3.26 (3.14)	7.76 (7.53)	57.6 (57.4)	3.71 (3.47)	4.05 (4.28)	12.5 (12.8)
100 kbp, 5%	**5.10 (4.77)**	8.45 (8.11)	14.9 (14.7)	14.4 (14.8)	146 (146)	63.3 (63.2)	55.0 (51.2)	24.0 (23.8)
100 kbp, 10%	**7.28 (7.02)**	13.9 (13.3)	27.6 (27.5)	48.3 (45.8)	247 (247)	218 (219)	182 (170)	39.0 (38.9)
100 kbp, 20%	**15.7 (15.6)**	36.7 (36.0)	56.8 (56.6)	t.o.	446 (446)	695 (695)	551 (510)	62.8 (62.7)
500 kbp, 1%	**33.5 (32.3)**	42.7 (41.3)	72.3 (72.2)	41.2 (39.2)	670 (670)	80.8 (79.4)	74.8 (63.8)	124 (124)
500 kbp, 5%	**31.4 (30.2)**	45.0 (44.4)	214 (213)	83.1 (82.5)	2,280 (2,280)	1,560 (1,560)	1,480 (1,250)	401 (400)
500 kbp, 10%	**68.4 (64.3)**	119 (117)	425 (424)	406 (406)	4,380 (4,380)	5,450 (5,450)	5,050 (4,250)	711 (711)
500 kbp, 20%	**255 (257)**	545 (545)	901 (896)	t.o.	8,520 [3/4] (8,520)	18,000 [1/4] (18,000)	o.o.m.	1,240 (1,240)
1 Mbp, 1%	**59.7 (56.8)**	89.1 (87.0)	253 (253)	97.3 (93.3)	2,380 (2,380)	366 (364)	391 (353)	398 (398)
1 Mbp, 5%	**65.0 (62.8)**	118 (117)	818 (815)	183 (182)	8,670 [3/4] (8,670)	6,350 (6,350)	o.o.m.	1,440 (1,440)
1 Mbp, 10%	**183 (183)**	369 (366)	1,650 (1,640)	978 (980)	17,300 [1/4] (17,300)	22,200 [1/4] (22,200)	o.o.m.	2,650 (2,650)
1 Mbp, 20%	**892 (688)**	2,020 (2,010)	3,520 (3,500)	t.o.	t.o.	t.o.	o.o.m.	4,800 (4,800)

Table S6: The samples: pairs in each sample of the dataset, their lengths and their divergence, the edit distance over the longer sequence, as QuickEd's Table S1 describes its datasets.

Dataset	Pairs	Length, min / mean / max (bp)	Divergence, min / mean / max (%)
Real data			
ont-1k	1221 of 12477	42 / 830 / 1,089	0 / 11 / 21
ont-10k	277 of 5000	342 / 3,643 / 9,791	4 / 13 / 20
ont-50k	104 of 10000	360 / 9,633 / 42,626	4 / 12 / 19
ont-500k	4 of 50	553,834 / 642,465 / 683,520	4 / 7 / 13
ont-500k-genvar	4 of 48	594,436 / 662,980 / 861,994	5 / 7 / 10
SARS-CoV-2	33 of 10000	29,244 / 29,746 / 29,864	0 / 1 / 7
Illumina	4032 of 5992	77 / 248 / 251	0 / 0 / 10
PacBio HiFi	69 of 890	10,956 / 14,372 / 22,816	0 / 0 / 1
Simulated: random sequences, uniform errors (QuickEd)			
10 kbp, 1%	100 of 10000	10,000 / 10,003 / 10,023	1 / 1 / 1
10 kbp, 5%	99 of 10000	10,000 / 10,008 / 10,045	5 / 5 / 5
10 kbp, 10%	100 of 10000	10,000 / 10,008 / 10,056	9 / 9 / 9
10 kbp, 20%	99 of 10000	10,000 / 10,014 / 10,095	17 / 17 / 17
100 kbp, 1%	9 of 1000	100,000 / 100,013 / 100,035	1 / 1 / 1
100 kbp, 5%	9 of 1000	100,000 / 100,028 / 100,123	5 / 5 / 5
100 kbp, 10%	10 of 1000	100,000 / 100,039 / 100,120	9 / 9 / 9
100 kbp, 20%	10 of 1000	100,000 / 100,040 / 100,155	17 / 17 / 17
500 kbp, 1%	4 of 200	500,000 / 500,038 / 500,103	1 / 1 / 1
500 kbp, 5%	4 of 200	500,016 / 500,088 / 500,178	5 / 5 / 5
500 kbp, 10%	4 of 200	500,000 / 500,052 / 500,105	9 / 9 / 9
500 kbp, 20%	4 of 200	500,000 / 500,055 / 500,220	17 / 17 / 17
1 Mbp, 1%	4 of 100	1,000,000 / 1,000,034 / 1,000,107	1 / 1 / 1
1 Mbp, 5%	4 of 100	1,000,185 / 1,000,301 / 1,000,551	5 / 5 / 5
1 Mbp, 10%	4 of 100	1,000,000 / 1,000,054 / 1,000,120	9 / 9 / 9
1 Mbp, 20%	4 of 100	1,000,000 / 1,000,166 / 1,000,355	17 / 17 / 17

Table S7: Memory at unit costs: the largest growth of peak resident memory aligning one pair of the sample, in MiB, as in A*PA2, and the peak resident memory of the whole run in parentheses, as in QuickEd, runtime and input included. Brackets give the pairs completed where not all were, with no whole-run peak; t.o. marks a tool that completed no pair within 30 s.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	Edlib	BiWFA	QuickEd
Real data						
ont-500k	48 (64)	82 (89)	82 (90)	5.3 (15)	15 (29)	22 (26)
ont-500k-genvar	39 (64)	48 (57)	32 (55)	4.8 (16)	12 (27)	20 (27)
Simulated: random sequences, uniform errors (QuickEd)						
100 kbp, 20%	5.2 (18)	7.3 (12)	4.7 (9)	1.4 (8)	3.6 (11)	8.8 (12)
500 kbp, 20%	53 (82)	59 (66)	53 (61)	4.1 [3/4]	15 [1/4]	16 (20)
1 Mbp, 20%	123 (178)	184 (196)	192 (205)	t.o.	t.o.	26 (31)

Table S8: Global scores of 500,000 short-read pairs: each time is the whole call, in ms.

Costs	Threads	dinara-align	WFA2-lib	KSW2	parasail	Edlib
Affine (1, 2, 1)	1	**230**	1,060	10,900	21,700	–
Unit	1	**212**	450	–	–	2,000
Affine (1, 2, 1)	10	**27.0**	116	1,210	2,320	–
Unit	10	**29.0**	64.0	–	–	216